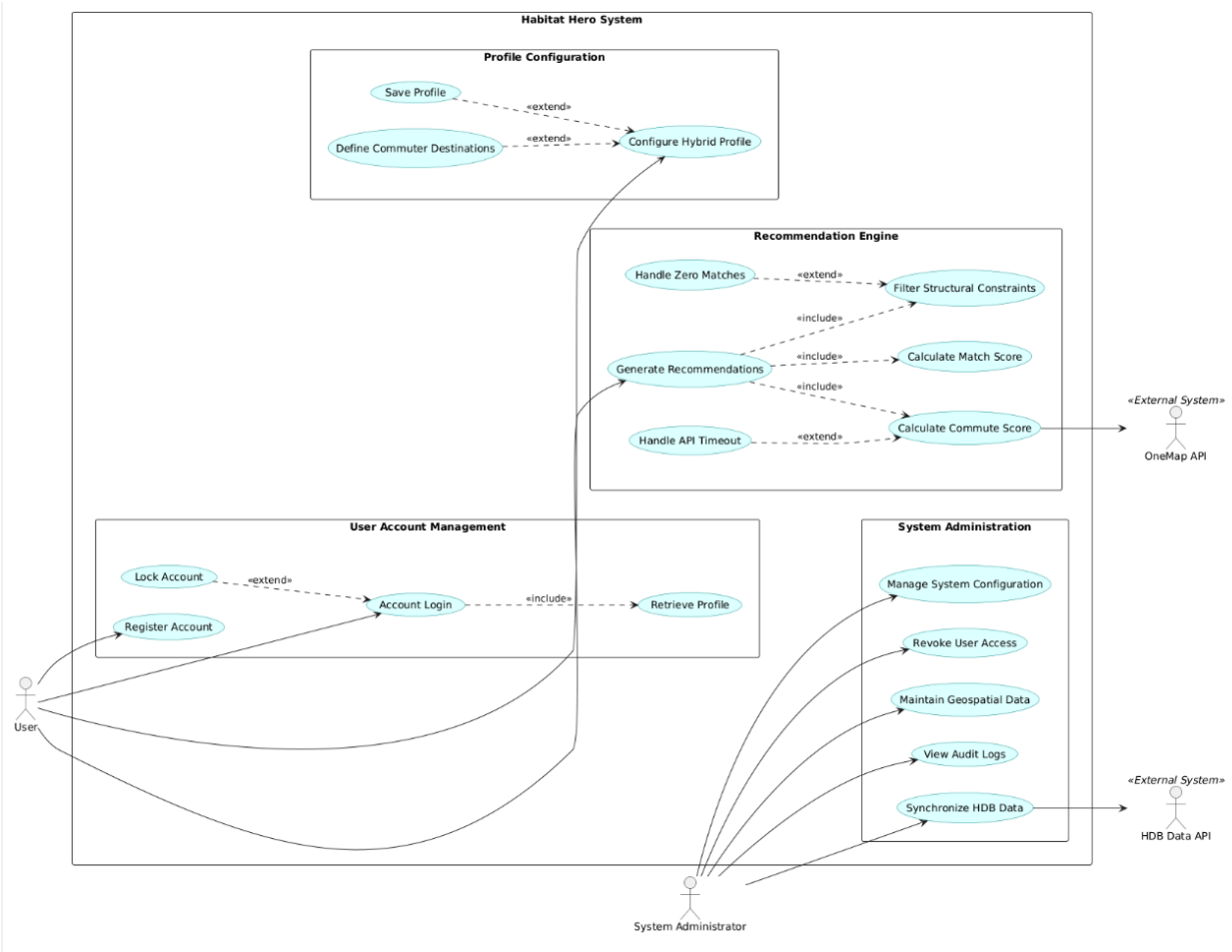
Table of Contents

1. Updated Requirements Model **2**
2. System Architecture **3**
2.1 Architectural Overview 4
2.2 Tech Stack Justification 5
3. Design Model **6**
4. Key Design Issues **7**
4.1 Data Persistence 7
4.2 Access Controls 7
4.3 Design Patterns 8
5. AI-Based Project Initialisation (Section 3.3.5 PDF Report) **9**
● In a separate PDF 9
6. Implementation & Collaboration **9**
● 6.1 Application Skeleton: Description of the manually modified folder structure and skeleton code. 9

1. Updated Requirements Model

- **1.1 Complete Use Case Diagram:** (Refined from Lab 2).



- **1.2 Use Case Descriptions:** Detailed text for all use cases.

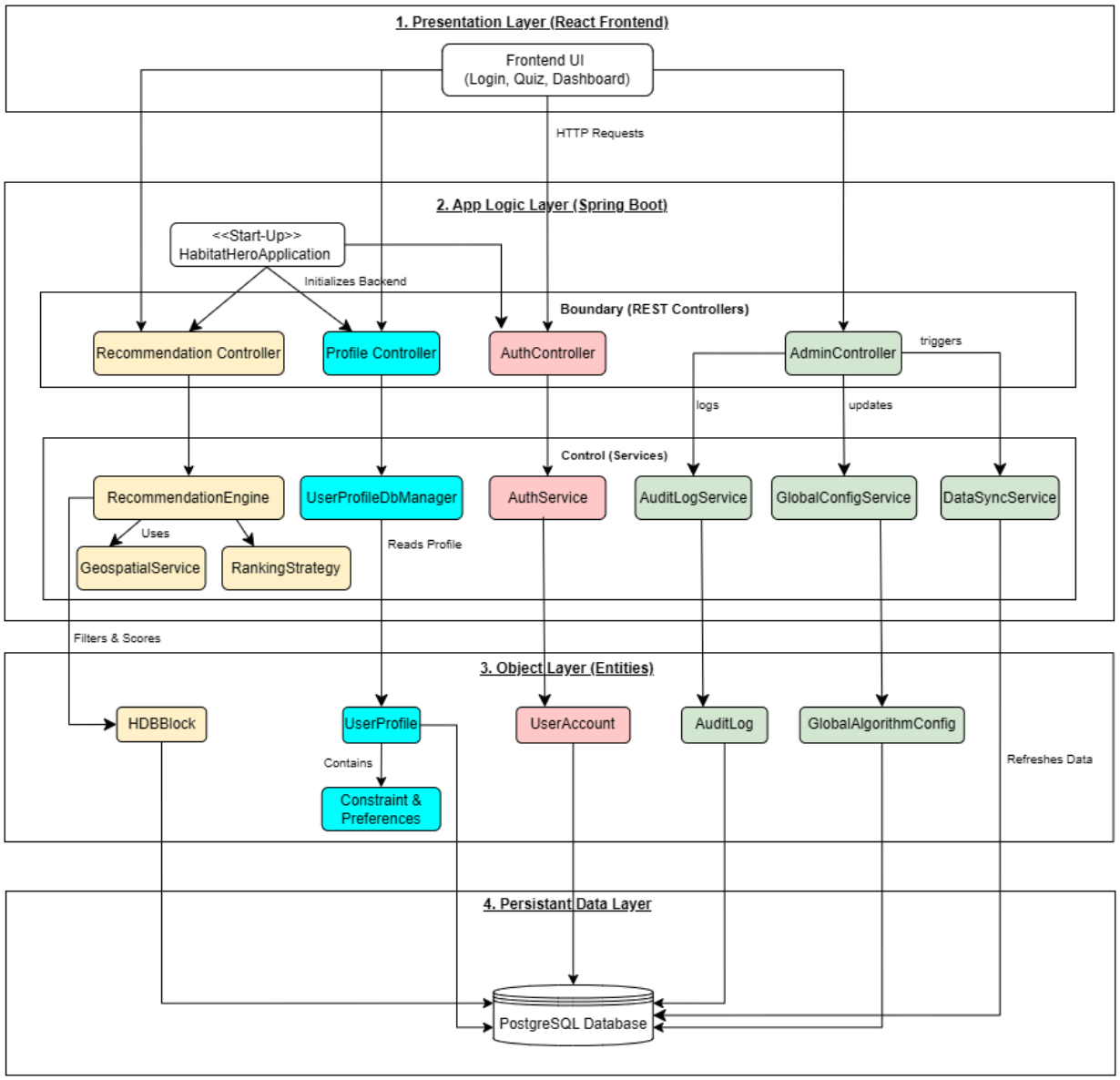
The Habitat Hero System is composed of five major functional groups that work together to translate user constraints and lifestyle preferences into a ranked list of housing recommendations. The major functions the product must perform include:

- **Hybrid User Profiling**
  - Allows users to define non-negotiable structural constraints, such as budget limits, preferred towns, and minimum lease duration.
  - Enables users to select livability factors (e.g., Solar Orientation, Acoustic Comfort) and toggle them as either strict requirements or weighted preferences.
  - Provides multi-commuter settings to input two destination addresses for joint household travel analysis.
- **Recommendation Engine Logic**

    - Executes a "Hard Filtering" process to immediately exclude HDB blocks that violate structural constraints or strict requirements.
    - Performs "Soft Scoring" by calculating attribute scores for the remaining blocks based on the user's weighted preferences.
    - Queries the OneMap Routing API to calculate a Commute Fairness Score and Total Commute Efficiency based on public transport travel times.
    - Aggregates all scores into a Global Match Index to generate and display a ranked list of property recommendations.
- **Geospatial Analysis Features**
    - Calculates building footprint polygons to identify and flag units with high West Sun heat exposure.
    - Generates 100-meter spatial proximity buffers around transport lines (MRT tracks and expressways) to determine noise risk.
    - Overlays URA Master Plan zoning data to alert users of future development risks, such as proximity to undeveloped Reserve Sites.
- **User Account Management**
    - Supports user registration and secure login authentication mechanisms, enforcing a 15-minute account lockout policy after 5 consecutive failed login attempts.
    - Allows authenticated users to securely save their active Hybrid User Profiles to the database for persistence, utilizing JWT (JSON Web Token) session extraction to guarantee data ownership and prevent unauthorized modifications.
    - Automatically retrieves and populates previously saved preferences upon user login.
- **System Administration**
    - Provides secure, headless RESTful API endpoints for managing backend operations.
    - Automates and manually triggers the synchronization of external datasets (e.g., Data.gov.sg HDB Resale transactions) to maintain data currency.
    - Allows administrators to manually trigger the background computation of spatial attributes (e.g., West Sun azimuth, noise buffers) to maintain geospatial data.
    - Allows administrators to dynamically tune global algorithm weighting parameters (e.g., noise penalties) without system recompilation.
    - Maintains system integrity by enabling user access revocation to suspend or delete specific accounts.
    - Provides an endpoint for administrators to retrieve and view a chronological, immutable audit log of system actions and events.

2. System Architecture

2.1 Architectural Overview



2.2 Tech Stack Justification

1. Presentation Layer: React (Frontend)

- **Component-Based Architecture:** React allows for the creation of reusable UI components (e.g., Login, Quiz, and Dashboard views), which promotes maintainability and a clean separation of presentation logic from data.
- **Single Page Application (SPA) Performance:** For an HDB recommendation tool, React provides a seamless, highly reactive user experience by updating search results dynamically without refreshing the page, which is essential for user engagement during the preference quiz.
- **State Management:** It efficiently handles complex user preference states, ensuring that data passed via HTTP requests to the backend is accurate and synchronized.

2. App Logic Layer: Spring Boot (Backend)

- **Seamless BCE Mapping:** Spring Boot facilitates the strict mapping of UML classes to Java classes required by the manual. Its structure naturally supports the Boundary-Control-Entity (BCE) pattern by using `@RestController` for Boundaries and `@Service` for Control logic.
- **RESTful API Integration:** Spring Boot simplifies the creation of robust REST endpoints (e.g., `RecommendationController`), enabling standard HTTP communication between the frontend and the backend logic.
- **Enterprise-Grade Robustness:** It provides built-in support for key design issues like Access Control (via Spring Security) and Persistence (via Spring Data JPA), ensuring the system is secure and scalable.

3. Persistent Data Layer: PostgreSQL (Database)

- **Relational Integrity:** As the housing domain involves complex, structured data (e.g., `HDBBlock`, `UserProfile`, and `GlobalAlgorithmConfig`), a relational database like PostgreSQL ensures data integrity and supports complex querying for the `RecommendationEngine`.
- **SQL Compatibility:** It provides a standardized interface for SQL queries triggered by the control services and entities (e.g., `UserProfileDbManager`), which allows for precise data retrieval and mapping to Java entities.

4. Architectural Choice: 4-Layered BCE Design

- **Loose Coupling and High Cohesion:** By transforming the analysis model into this 4-layer design, the team achieves loose coupling; the Presentation Layer can be modified without impacting the Object Layer or Persistent Data Layer.

- **Extensibility:** This architecture allows the system to remain "stable" for implementation while making it easy to add new features, such as new ranking strategies (`RankingStrategy`) in the Control layer, without breaking existing boundaries.
- **Traceability:** The chosen stack ensures clear traceability from the initial requirements to the final implementation, as every UML box in the design matches a physical file in the repository hierarchy.

3. Design Model

- **3.1 Design Class Diagram (DCD):** * Mapping Analysis classes to Design classes.
    - Inclusion of attributes, operations, and visibility (+/-).
    - Stereotypes (Boundary, Control, Entity).
- **3.2 Refined Dialog Map:**

  We need to

  - **Match File Names**: Rename generic screen boxes to match your React files exactly (e.g., ResultsPage.js «Boundary»).
  - **Label with Operations**: Arrows between screens must be labeled with actual method names like handleLogin() or submitQuiz().
  - **Show Data Passing**: Clearly write what data is moving between screens (e.g., [UserProfile] or [List<HDBBlock>]) on the transition lines.
  - **Add Access Control Logic**: Draw the "Unauthorized" redirect path (e.g., if a user isn't logged in, they get sent to LoginPage.js) to address security requirements.
  - **Include Conditions**: Use brackets for logic-based navigation, such as [if results == empty].

4. Key Design Issues

4.1 Data Persistence

The HabitatHero system utilizes a Layered Architecture to manage persistent data, ensuring that the domain logic remains independent of the database implementation.

Database Choice	The system is designed to interface with a PostgreSQL relational database, which provides the robust ACID compliance necessary for handling sensitive user profiles and financial HDB resale data.
Persistence Strategy	Data access is managed through the Entity Manager/Repository pattern, primarily via the UserProfileDbManager and ResaleTransactionService
Schema Overview	The persistent storage is structured around several key Entity classes: ● UserAccount and UserProfile: Stores authentication credentials and complex preference objects such as WeightedPreference and StructuralConstraints. ● HDBBlock and ResaleTransaction: Contains large-scale datasets for housing blocks and historical transaction records used by the recommendation engine. ● AuditLog: Maintains a persistent record of administrative actions for security and tracking
Implementation	A Singleton pattern is applied to the UserProfileDbManager to ensure a single, consistent gateway to the database, preventing connection leaks and data inconsistency.

4.2 Access Controls

To satisfy the requirement for providing access control, the system implements a robust authentication and authorization framework within the App Logic layer

User Roles	The system distinguishes between two primary actors: the User, who accesses recommendation features, and the System Administrator, who manages global configurations and data refreshes.
Security Boundary	All external requests are intercepted by the `AuthController` and `AdminController`, which act as the boundary between the React frontend and the protected service logic.
Authentication Mechanism	The `AuthService` validates credentials against the `UserAccount` entity. Upon successful login, the system typically issues a JWT (JSON Web Token) to maintain a stateless session, ensuring that only authenticated

	users can access endpoints like `getRecommendations` or `updateUserProfile`
Authorization Logic	The `AdminController` explicitly restricts sensitive operations such as `triggerDataRefresh()` and `updatePenaltyMultiplier()` to verified administrator accounts, protecting the integrity of the ranking algorithm and global configurations

4.3 Design Patterns

The design model incorporates several object-oriented patterns to improve extensibility, maintainability, and loose coupling

Strategy Pattern	Applied to the ranking logic via the `RankingStrategy` interface. This allows the `RecommendationEngine` to switch between different algorithms such as `BudgetRanker`, `CommuteRanker`, or `LivabilityRanker` at runtime without modifying the core engine code, facilitating easy addition of new ranking criteria in the future.
Singleton Pattern	Implemented in the `UserProfileDbManager`. By using a private constructor and a static `getInstance()` method, the system ensures that only one instance of the database manager exists, which is critical for managing shared resources like database connection pools
BCE (Boundary-Control-Entity) Pattern	The overall system follows the BCE heuristic to separate concerns. **Boundary** classes (Controllers) handle communication, **Control** classes (Services/Engines) coordinate business logic, and **Entity** classes represent the data model, ensuring high cohesion within each layer
Start-up Heuristic	The `HabitatHeroApp` class is designated as the `«start-up»` class, responsible for system initialization and the orchestration of backend and frontend services during project launch

5. AI-Based Project Initialisation (Section 3.3.5 PDF Report)

- **In a separate PDF**

6. Implementation & Collaboration

- **6.1 Application Skeleton: Description of the manually modified folder structure and skeleton code.**